1-3

LangChain 프레임워크 깊이 있는 학습

학습 목차

- **LCEL과 Runnable**
  - LCEL의 설계 철학
  - Runnable 인터페이스
- **Tool Calling**
  - Tool Schema와 도구 선택
  - 응답 파싱과 실행 흐름
- **구조화된 출력과 메모리**
  - Output Parser와 Pydantic
  - Buffer, Summary, VectorStoreRetriever Memory

© Scopelabs. All Rights Reserved

LCEL과 Runnable

LCEL의 설계 철학 (1)

LCEL은 LLM 애플리케이션을 작은 실행 단위의 조합으로 표현하기 위한 LangChain의 체인 구성 방식이다.

- 프롬프트, 모델, 파서, 함수 등을 하나의 흐름으로 연결한다.
- `|` 연산자는 앞 단계의 출력을 다음 단계의 입력으로 전달한다.
- 복잡한 Agent도 결국 입력을 변환하고 다음 실행 단위로 넘기는 흐름으로 이해할 수 있다.

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template(
    "{topic}을 초보자에게 설명해줘."
)
model = ChatOpenAI(model="gpt-4.1-mini")
parser = StrOutputParser()

chain = prompt | model | parser
```

LCEL의 설계 철학 (2)

LCEL의 핵심 장점은 실행 흐름을 선언적으로 읽을 수 있다는 점이다.

- 코드가 데이터 처리 파이프라인처럼 보인다.
- 각 단계의 역할과 연결 관계가 명확해진다.
- 동기, 비동기, 스트리밍, 배치 실행 같은 실행 방식을 같은 구조에서 다루기 쉽다.

Runnable 인터페이스 (1)

Runnable은 LangChain에서 실행 가능한 구성 요소를 공통 방식으로 다루기 위한 인터페이스이다.

- 프롬프트 템플릿, LLM, 출력 파서, 사용자 정의 함수가 모두 Runnable처럼 연결될 수 있다.
- `invoke`는 하나의 입력에 대해 하나의 결과를 반환한다.
- `batch`, `stream`, `ainvoke` 같은 실행 방식으로 같은 체인을 다양한 환경에서 사용할 수 있다.

```
result = chain.invoke({"topic": "LCEL"})
print(result)

results = chain.batch([
    {"topic": "Runnable"},
    {"topic": "Tool Calling"},
])

for chunk in chain.stream({"topic": "Agent"}):
    print(chunk, end="")
```

Runnable 인터페이스 (2)

Runnable 조합은 복잡한 입력 변환을 단계별로 나누는 데 유용하다.

1. `RunnablePassthrough`는 입력을 그대로 다음 단계로 전달한다.
2. `RunnableLambda`는 일반 함수를 체인 안의 실행 단위로 감싼다.
3. `RunnableParallel`은 여러 실행 결과를 병렬로 만들어 하나의 입력 묶음으로 합친다.

이 조합을 통해 검색 결과, 사용자 질문, 메타데이터를 하나의 프롬프트 입력으로 구성할 수 있다.

Tool Calling

Tool Schema와 도구 선택 (1)

Tool Calling은 LLM이 자연어 답변 대신 사용할 도구와 인자를 구조화된 형태로 선택하게 하는 방식이다.

- Tool Schema는 도구 이름, 설명, 입력 인자, 인자 타입을 모델에게 알려준다.
- 모델은 사용자의 요청과 도구 설명을 비교해 어떤 도구가 필요한지 판단한다.
- 좋은 도구 설명은 도구의 기능뿐 아니라 언제 사용해야 하는지를 분명히 말해야 한다.
  - 도구 이름은 짧고 목적이 드러나도록
  - 인자 설명은 모델이 값을 잘 채울 수 있을만큼 구체적으로
  - 유사한 도구가 많으면 선택 기준 추가

응답 파싱과 실행 흐름 (1)

Tool Calling의 실행은 모델이 도구를 직접 실행하는 것이 아니라, 모델의 도구 호출 의도를 외부 런타임이 실행하는 구조이다.

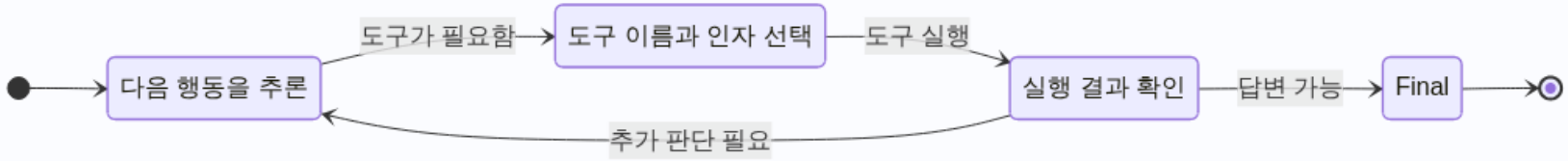
- 모델은 호출할 도구 이름과 인자 값을 생성한다.
- 애플리케이션은 그 결과를 파싱해 실제 함수를 실행한다.
- 함수 실행 결과는 다시 모델 입력으로 전달되어 최종 답변 생성에 사용된다.

```
{
  "tool": "get_weather",
  "arguments": {
    "location": "Seoul",
    "date": "2026-06-14"
  }
}
```

응답 파싱과 실행 흐름 (2)

Agent의 기본 실행 흐름은 Tool Calling을 반복적으로 활용한다.

1. 사용자의 요청을 보고 필요한 행동을 판단한다.
2. 적절한 도구와 입력 인자를 선택한다.
3. 도구 실행 결과를 관찰한다.
4. 추가 행동이 필요하면 다시 도구를 선택하고, 충분하면 최종 답변을 생성한다.



구조화된 출력과 메모리

Output Parser와 Pydantic

Output Parser는 LLM의 텍스트 출력을 애플리케이션이 사용할 수 있는 구조화된 데이터로 바꾸는 역할을 한다.

- 일반 텍스트 답변은 사람이 읽기에는 좋지만 프로그램이 안정적으로 처리하기 어렵다.
- JSON, 리스트, 객체 형태로 출력 규칙을 정하면 후속 로직이 단순해진다.
- Parser는 모델 출력이 약속한 형식에 맞는지 확인하고 필요한 데이터만 추출한다.

Pydantic은 구조화된 출력의 스키마와 검증 규칙을 코드로 표현하는 데 자주 사용된다.

- 필드 이름, 타입, 설명을 모델에게 명확히 전달할 수 있다.
- 잘못된 타입이나 누락된 필드를 검증할 수 있다.
- 평가, 라우팅, 저장, UI 표시처럼 정해진 데이터 구조가 필요한 작업에 적합하다.

Buffer, Summary, VectorStoreRetriever Memory

Memory는 대화나 작업 과정에서 이전 정보를 다음 요청에 반영하기 위한 설계이다.

- Buffer Memory는 이전 대화를 그대로 보관해 모델에게 전달한다.
- Summary Memory는 긴 대화를 요약해 컨텍스트 사용량을 줄인다.
- VectorStoreRetriever Memory는 과거 기록을 임베딩해 관련성이 높은 기억만 검색한다.

메모리 전략은 정확도, 비용, 지연시간의 균형으로 선택한다.

- 짧은 대화는 Buffer Memory가 단순하고 정확하다.
- 긴 대화는 Summary Memory로 핵심 맥락을 압축한다.
- 누적 지식이 많은 서비스는 VectorStoreRetriever Memory로 필요한 기억만 가져온다.

중요한 상태는 자연어 기억보다 별도 데이터 구조로 저장하는 편이 안정적이다.